

Track C — Configurable Options Reference

This document lists **every configurable parameter** available in Track C (RGTP Token Pruning) for running different experiments. Track C applies **Run-time Guided Token Pruning (RGTP)** on top of a trained Track B checkpoint to evaluate inference efficiency vs. accuracy tradeoffs.

Table of Contents

1. [YAML Config File \(configs/trackC.yaml \)](#)
 2. [RGTP \(Pruning\) Configuration](#)
 3. [Evaluation Configuration](#)
 4. [Instrumentation Configuration](#)
 5. [Rate Sweep Configuration](#)
 6. [CLI Arguments](#)
 7. [RuntimeConfig \(In-Code Toggles\)](#)
 8. [Quick Reference Tables](#)
 9. [Output Artifacts](#)
-

1. YAML Config File

File: `local_extraction/configs/trackC.yaml`

The main configuration file for Track C. All settings can be modified here.

Inheritance

```
_base_: "base.yaml" # Inherits shared settings from base.yaml
```

Inherited settings from `base.yaml` :

- `version` : Data version (default: `"v2"`)
 - `paths.*` : Common path definitions
 - `runtime.*` : Runtime settings (num_workers, print_progress)
 - `demo.*` : Demo/smoke test settings
-

2. RGTP (Pruning) Configuration

RGTP (Rollout-Guided Token Pruning) is the core contribution of Track C. It discards low-importance candidate tokens before the head for inference efficiency.

Main RGTP Settings

```
rgtp:
  enabled: true
  rate: 0.1
  min_keep: 2
  temporal_decay: 0.6
  logit_fill: -12.0
```

Parameter	Default	Range	Description
enabled	true	true/false	Enable/disable RGTP pruning
rate	0.1	0.0 - 0.95	Fraction of tokens to drop (0.0 = no pruning, 0.5 = drop 50%)
min_keep	2	≥1	Minimum candidates to keep per frame (safety floor)
temporal_decay	0.6	0.0 - 1.0	Blend factor: <code>rollout * decay + motion * (1 - decay)</code>
logit_fill	-12.0	Large negative	Logit value for pruned tokens (ensures low probability)

RGTP Rate Examples

Rate	Effect	Use Case
0.0	No pruning (baseline)	Track B comparison
0.1	Drop 10% tokens	Minimal pruning
0.3	Drop 30% tokens	Moderate efficiency
0.5	Drop 50% tokens	Aggressive pruning
0.7	Drop 70% tokens	Maximum efficiency

Importance Scoring Algorithm

The RGTP importance score for each candidate is computed as:

importance = temporal_decay * rollout + (1 - temporal_decay) * motion

Where:

- rollout**: FGTP attention scores from frame t-1 (saliency from fusion)
- motion**: Energy of token change from t-1 to t (motion-aware adjustment)

Candidates with the lowest importance scores are pruned up to `rate` fraction.

3. Evaluation Configuration

Settings for loading data and running evaluation.

Checkpoint & Manifest Paths

```
evaluation:
  checkpoint: null      # Path to Track B checkpoint (null = auto-detect
latest)
  stageB_run: null     # Path to Stage B run directory (null = auto-detect)
  val_manifest: null   # Path to validation manifest (null = auto-detect)
```

Setting	Description
checkpoint: null	Auto-detect latest trackB_final_*.pt or trackB_best*.pt
checkpoint: "path/to/ckpt.pt"	Use specific checkpoint
stageB_run: null	Auto-detect latest Track A Stage B run
val_manifest: null	Auto-detect from stageB_run or manifests directory

Batch Settings

```
evaluation:
  batch_size: 1        # Batch size (typically 1 for RGTP streaming)
  candidate_limit: 16   # Max candidates per frame
  normalize_ttc: true   # Use normalized TTC values
```

Parameter	Default	Description
batch_size	1	RGTP typically runs bs=1 for per-frame pruning
candidate_limit	16	Maximum candidates per frame (from Track B config)
normalize_ttc	true	Normalize TTC using dataset statistics

4. Instrumentation Configuration

Settings for profiling latency, VRAM, and FLOPs.

```
instrumentation:
  enabled: true
  record_latency: true
  record_vram: true
  record_flops: true
  use_cuda_events: true
  warmup_iters: 1
  bench_iters: 5
  bench_samples: 1
```

Parameter	Default	Description
enabled	true	Master toggle for all instrumentation
record_latency	true	Record per-sample latency (ms)
record_vram	true	Record peak CUDA VRAM usage
record_flops	true	Profile head forward pass FLOPs
use_cuda_events	true	Use CUDA events for precise timing (falls back to wall clock)
warmup_iters	1	Warmup iterations before micro-benchmark
bench_iters	5	Iterations for micro-benchmark timing
bench_samples	1	Number of samples to micro-benchmark

Runtime Metrics Collected

When instrumentation is enabled, the following metrics are recorded:

Metric	Description
latency_ms_mean	Mean latency per sample (ms)
latency_ms_median	Median latency per sample (ms)
latency_ms_p90	90th percentile latency (ms)
latency_ms_p95	95th percentile latency (ms)
throughput_samples_per_s	Samples processed per second
throughput_candidates_per_s	Candidates processed per second
head_benchmark_ms_mean	Mean head forward pass time (ms)
peak_vram_bytes	Peak CUDA memory allocated
peak_vram_reserved_bytes	Peak CUDA memory reserved
head_flops	FLOPs for head forward pass

5. Rate Sweep Configuration

Automated sweep over multiple pruning rates to generate Pareto curves.

```
rate_sweep:
  enabled: false
  rates: [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6]
```

Parameter	Default	Description
enabled	false	Enable rate sweep mode
rates	[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6]	List of pruning rates to test

Typical Rate Sweep Values

```
# Fine-grained sweep
rates: [0.0, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5]

# Coarse sweep
rates: [0.0, 0.2, 0.4, 0.6]

# Extended sweep
rates: [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]
```

6. CLI Arguments

Track C supports command-line arguments for quick parameter overrides.

```
python local_extraction/trackC/trackC_pruning.py [OPTIONS]
```

Argument	Type	Default	Description
--config	str	trackC	Config name to load
--video_backbone	str	null	Override video backbone (resnet18 or videomae_ego)
--rgtp_rate	float	null	Override RGTP pruning rate (0.0-0.95)
--checkpoint	str	null	Override checkpoint path
--no_pruning	flag	false	Disable pruning (baseline mode)

CLI Usage Examples

```
# Default (uses trackC.yaml)
python local_extraction\trackC\trackC_pruning.py

# Baseline (no pruning)
python local_extraction\trackC\trackC_pruning.py --no_pruning

# Specific pruning rate
python local_extraction\trackC\trackC_pruning.py --rgtp_rate 0.3

# 50% pruning
python local_extraction\trackC\trackC_pruning.py --rgtp_rate 0.5

# With specific checkpoint
python local_extraction\trackC\trackC_pruning.py --checkpoint
"local_extraction/runs/Track_B/checkpoints/trackB_best.pt"

# VideoMAE backbone
python local_extraction\trackC\trackC_pruning.py --video_backbone videomae_ego
```

7. RuntimeConfig (In-Code Toggles)

The `RuntimeConfig` class in `trackC_pruning.py` provides additional in-code toggles:

```
class RuntimeConfig:
    # Path overrides (None = auto-discover)
    checkpoint: Optional[str] = None
    stageB_run: Optional[str] = None
    val_manifest: Optional[str] = None

    # Pruning behaviour
    pruning_enabled: bool = True
    rgtp_rate: float = 0.1
    min_keep: int = 2

    # Instrumentation toggles
    measure_latency: bool = True
    measure_vram: bool = True
    measure_flops: bool = True
    bench_warmup: int = 1
    bench_iters: int = 5
    bench_samples: int = 1
```

These can be modified directly in the code or via YAML config.

8. Quick Reference Tables

RGTP Parameters

Parameter	Location	Values	Impact
<code>enabled</code>	<code>rgtp.enabled</code>	<code>true/false</code>	Enable/disable RGTP
<code>rate</code>	<code>rgtp.rate</code>	<code>0.0-0.95</code>	Fraction of tokens to drop
<code>min_keep</code>	<code>rgtp.min_keep</code>	<code>≥1</code>	Minimum candidates per frame
<code>temporal_decay</code>	<code>rgtp.temporal_decay</code>	<code>0.0-1.0</code>	Rollout vs motion blend
<code>logit_fill</code>	<code>rgtp.logit_fill</code>	Large negative	Pruned token logit value

Evaluation Parameters

Parameter	Location	Values	Impact
<code>checkpoint</code>	<code>evaluation.checkpoint</code>	Path or null	Track B model to evaluate
<code>stageB_run</code>	<code>evaluation.stageB_run</code>	Path or null	Stage B candidates source
<code>val_manifest</code>	<code>evaluation.val_manifest</code>	Path or null	Validation manifest path
<code>batch_size</code>	<code>evaluation.batch_size</code>	<code>1</code> (typical)	Batch size for evaluation
<code>candidate_limit</code>	<code>evaluation.candidate_limit</code>	<code>1-32</code>	Max candidates per frame

Instrumentation Parameters

Parameter	Location	Values	Impact
<code>enabled</code>	<code>instrumentation.enabled</code>	<code>true/false</code>	Master toggle
<code>record_latency</code>	<code>instrumentation.record_latency</code>	<code>true/false</code>	Per-sample timing
<code>record_vram</code>	<code>instrumentation.record_vram</code>	<code>true/false</code>	CUDA memory tracking
<code>record_flops</code>	<code>instrumentation.record_flops</code>	<code>true/false</code>	FLOPs profiling
<code>use_cuda_events</code>	<code>instrumentation.use_cuda_events</code>	<code>true/false</code>	Precise CUDA timing
<code>warmup_iters</code>	<code>instrumentation.warmup_iters</code>	<code>0-5</code>	Benchmark warmup
<code>bench_iters</code>	<code>instrumentation.bench_iters</code>	<code>1-10</code>	Benchmark iterations

CLI Arguments Summary

Argument	Values	Impact
<code>--config</code>	Config name	Load different config file

Argument	Values	Impact
--video_backbone	resnet18 , videomae_ego	Override backbone
--rgtp_rate	0.0-0.95	Override pruning rate
--checkpoint	Path	Override checkpoint
--no_pruning	Flag	Disable pruning (baseline)

9. Output Artifacts

All outputs are saved under `local_extraction/runs/Track_C/` :

Metrics Files

```
metrics/  
  trackC_val_rate<XX>_<timestamp>.json      # Main metrics  
  trackC_val_rate<XX>_<timestamp>_summary.json # Full summary with configs
```

Metrics JSON Contents

```
{  
  "accuracy": 0.85,  
  "mAP": 0.3565,  
  "ttc_mae_seconds": 0.75,  
  "num_candidates": 1234,  
  
  // RGTP Stats  
  "rgtp_enabled": true,  
  "rgtp_rate_request": 0.3,  
  "rgtp_mean_fraction_pruned": 0.28,  
  
  // Semantic Metrics (N/N+V/N+δ/All)  
  "N_mAP": 0.42,  
  "Nv_mAP": 0.38,  
  "N_delta_mAP": 0.35,  
  "All_mAP": 0.32,  
  
  // Top-5 Metrics  
  "N_top5_acc": 0.65,  
  "Nv_top5_acc": 0.58,  
  "N_delta_top5_acc": 0.52,  
  "All_top5_acc": 0.48,  
  "N_top5_mAP": 0.55,  
  "Nv_top5_mAP": 0.50,  
  
  // Runtime Metrics (when instrumentation enabled)
```

```
"latency_ms_mean": 12.5,  
"latency_ms_median": 11.8,  
"latency_ms_p90": 15.2,  
"latency_ms_p95": 18.1,  
"throughput_samples_per_s": 80.0,  
"throughput_candidates_per_s": 640.0,  
"peak_vram_bytes": 1234567890,  
"head_flops": 1.2e9,  
  
// Per-class breakdowns  
"per_noun_stats": {...},  
"per_verb_stats": {...}  
}
```

Plots (from `trackC_plots.py`)

```
plots/  
  trackC_metrics_over_time.png    # Metrics across runs  
  trackC_metrics_summary.tsv      # Tab-separated summary
```

Example Experiment Configurations

1. Baseline (No Pruning)

```
rgtp:  
  enabled: false  
  rate: 0.0
```

Or via CLI:

```
python local_extraction\trackC\trackC_pruning.py --no_pruning
```

2. Light Pruning (10%)

```
rgtp:  
  enabled: true  
  rate: 0.1  
  min_keep: 3
```

3. Moderate Pruning (30%)

```
rgtp:  
  enabled: true  
  rate: 0.3  
  min_keep: 2
```

4. Aggressive Pruning (50%)

```
rgtp:  
  enabled: true  
  rate: 0.5  
  min_keep: 2
```

5. Maximum Efficiency (70%)

```
rgtp:  
  enabled: true  
  rate: 0.7  
  min_keep: 1
```

6. Rate Sweep Experiment

```
rate_sweep:  
  enabled: true  
  rates: [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6]
```

7. Motion-Heavy Importance

```
rgtp:  
  enabled: true  
  rate: 0.3  
  temporal_decay: 0.3 # More weight on motion energy
```

8. Rollout-Heavy Importance

```
rgtp:
  enabled: true
  rate: 0.3
  temporal_decay: 0.9 # More weight on attention rollout
```

9. Full Instrumentation

```
instrumentation:
  enabled: true
  record_latency: true
  record_vram: true
  record_flops: true
  use_cuda_events: true
  warmup_iters: 2
  bench_iters: 10
  bench_samples: 3
```

10. Quick Smoke Test

```
smoke_test:
  enabled: true
  max_samples: 5
  test_rates: [0.0, 0.3]
```

Files Reference

File	Purpose
configs/trackC.yaml	Main YAML configuration
trackC_pruning.py	Main RGTP evaluation script
trackC_compare_metrics.py	Compare Track B vs Track C metrics
trackC_plots.py	Generate metrics plots and TSV summaries
trackC_tests.py	Unit tests
README.md	Detailed pipeline guide

Comparison Tools

Compare Latest Track B vs Track C

```
python local_extraction\trackC\trackC_compare_metrics.py
```

Compare All Runs

```
python local_extraction\trackC\trackC_compare_metrics.py --all --out  
comparison.tsv
```

Generate Plots

```
python local_extraction\trackC\trackC_plots.py --runs_dir  
local_extraction/runs/Track_C --out_dir local_extraction/runs/Track_C/plots
```